User manual / 1 of 2

Repository https://github.com/SambhavMayani/ADSBDB
Authors Mario Rodrigo, Sambhav and Wassim.

These instructions use Windows and Git Bash, from the repository folder. Install 64-bit Python 3.11, Git and Docker Desktop first, then start Docker Desktop. You need Internet to install libraries and download models. Inference runs on CPU. No GPU or paid AI API is required. Linux instructions are in `README.md`.

1. Install the application and check the code

```
git clone https://github.com/SambhavMayani/ADSBDB.git
cd ADSBDB
python -m venv .venv
source .venv/Scripts/activate
python -m pip install -r requirements.txt
python -m pip check
python -m pytest -q
python -m orchestration.pipeline --plan
```

The tests should pass. The plan lists 16 steps without running them. Neither command needs your wildlife data or model weights. In each new terminal, return to this folder and activate `.venv` again. If the repository is private, the assessor needs access.

2. Download the pretrained models once

Libraries contain the code. Model weights are the learned parameters that the code loads to process an image, sound or text. Downloading them does not train a model or create dataset embeddings. Prepare the embedding models before running the pipeline.

```
python -m exploitation_zone.prepare_models --download
```

Expect `Embedding models ready`. For Task 3 only, also prepare Qwen and BLIP.

```
python -m multimodal_tasks.rag_model --download
HF_HUB_OFFLINE=0 python -c "from huggingface_hub import snapshot_download; \
from multimodal_tasks.generate import MODEL, REVISION; \
snapshot_download(repo_id=MODEL, revision=REVISION, token=False)"
```

Later runs reuse the local cache. The pipeline loads weights offline. Its acquisition option downloads *wildlife data*, not models. Task 3 checks for 2.5 GiB of available RAM before generation. This is a guard, not a full machine specification. Close other applications if needed and allow minutes for CPU responses. Keep free disk space for weights, data and the query copy.

3. Start local object storage

```
cp -n .env.example .env
# Edit .env: set your local storage username and password.
docker compose up -d
python orchestration/check_minio_connection.py
```

Set `MINIO_ROOT_USER` and `MINIO_ROOT_PASSWORD` before creating a new storage service. Keep existing credentials when using a prepared installation. Never commit `.env`. Port 9000 is the S3 API and port 9001 is the storage console. The EcoSentinel interface uses a different port, shown on the next page.

4. Choose the route that matches your data

A. Starting without local raw data. Set the provider credentials in `.env`, including `XENO_CANTO_API_KEY`, and review the species/acquisition settings in `config/`. Then run the following command.

```
python -m orchestration.pipeline --run --with-acquisition
```

This runs three download scripts and then the normal pipeline, making 19 steps in total. Providers can change content or impose quotas. The default audio request is four recordings per species, and shortages can stop acquisition. Inspect the log and keep successful downloads. If you decide to use the available smaller dataset, review its metadata and continue with route B. A fresh download need not reproduce our reference counts.

B. Raw data already supplied locally. Place the files and their matching `_metadata.json` records under `data/raw_dataset/<modality>/<species_id>/`. Use the supplied/acquisition layout, since arbitrary files without provenance are insufficient. Run the following command.

```
python -m orchestration.pipeline --run
```

Expect `Pipeline status: completed`. The last step checks that accepted files and vector segments agree. Logs and the run JSON are under `reports/orchestration/`.

C. Storage and Chroma already prepared. Keep their matching data and model cache. Skip rebuilding and run `python -m orchestration.reconcile`. Expect `completed` and an empty `problems` list. Only the unchanged reference installation should have 198 accepted files and 6,021 vectors. Git contains neither that dataset nor the weights.

5. Prepare searches and open the interface

Stop pipeline/index writers first. Use a new, nonexistent destination.

```
python -m orchestration.audit --source data/chromadb \
  --snapshot data/search_index
python -m multimodal_tasks.search --status
python -m webapp.server
```

The first command verifies and copies the existing database. It does not run encoders again or copy media. Chroma already persists vectors on disk, but opening it can cause internal writes. Our interface therefore opens this separate copy to protect the original. The copy costs extra disk space and does not update itself. Reuse it while the data is unchanged. After an index update, create a new copy and pass its path with `--index NEW_PATH` to the search/web commands.

Open http://127.0.0.1:8765. The pipeline does not start the web automatically. Try an image in *Buscar similares* for Task 1, a wildlife phrase in *Explorar por texto* with mixed results for Task 2, and a habitat/diet question in the RAG tab for Task 3. Check the sources. Search quantity defaults to eight files and can be set to 1–50. CLI searches use `--limit N`. Stop the server with Ctrl+C.

6. Inspect what the run produced

Open `reports/trusted_quality/<modality>/<timestamp>/quality_report.html`. Keep its PNG files beside it. The same folder contains JSON summaries and CSV measurements. Expected results and recovery steps for all routes are in `docs/user_guide.md`. A run that fails a check needs investigation before its results can be accepted.

EcoSentinel

Wildlife data, search and evidence

ADSDB P1
Universitat Politècnica de Catalunya

Mario Rodrigo
Sambhav
Wassim

4 November 2026

About this project

EcoSentinel brings wildlife photographs, recordings and descriptions into one local system. We preserve the original files, check their quality and make them searchable. Users can find related material and ask questions whose answers point back to retrieved sources.

Our main goal was to build a clear, reliable and testable pipeline. Wildlife gives the work a practical setting, but this project does not claim expert species recognition. The manual comes first so that a new reader can install the application, run the pipeline and inspect its results. Eight pages explain the design and evidence, followed by a separate query-flow appendix.

Purpose, data and architecture 1
Landing and Formatted 2
Trusted decisions and quality reports 3
Exploitation and model choices 4
Similarity search and its limits 5
Tasks 1 and 2, query preparation and interface 6
Task 3 / answers from retrieved evidence 7
Orchestration, validation and reproducibility 8
Query preparation and all three Tasks A1

1. Purpose, data and architecture

Why wildlife data?

A field team may need a photograph, an animal call and a description of its habitat, yet these materials often sit in separate catalogues. We chose Iberian fauna to explore how one system could connect them. Our priority for P1 was the code, pipeline design and validation, rather than optimising biological results. We still used the domain to guide decisions that would matter with more data and models.

EcoSentinel could help teachers prepare wildlife lessons or help analysts review material related to an observation. A user can start with a file or a phrase, find related records and follow an answer back to its source. The intended value is less manual searching across catalogues. We have not measured time savings or tested a commercial service. Reliable species identification would also require expert evaluation.

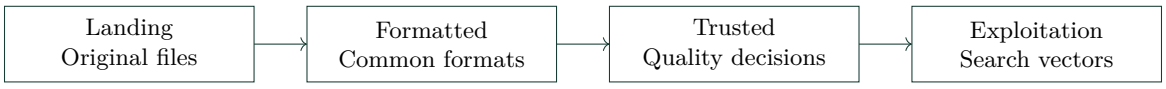
What the reference dataset contains

Type	Files	Sources
Images	100	Wikimedia Commons, ten images per species
Audio	34	Xeno-canto, recordings for nine species
Text	64	20 Wikipedia articles and 44 GBIF descriptions

The ten species include the Iberian lynx, wolf, deer, wild boar, eagle, eagle owl, kingfisher, robin, Perez's frog and ocellated lizard. Scientific identifiers in `config/species.yaml` link records across zones. Audio adds a useful ecological view, but coverage is uneven because the lizard has no recording and the lynx has only two. Text also varies by species and source. We keep these gaps visible instead of treating the sample as representative of Iberian biodiversity.

The acquisition scripts save URLs, provider identifiers and available licence/attribution fields beside each raw file. Wikipedia records also retain revision information. Provider labels help trace the data, but they are not expert checks of the species. Some audio licences restrict commercial reuse, so any future business use would need to review permissions for each asset.

Why four zones?



Verified query copy / Similarity search / RAG / Web interface

Each zone handles one concern, from arrival and format conversion to quality checks and search. This separation lets us change a quality rule or encoder without losing the original evidence. Python packages follow these boundaries. S3 object storage suits the large, varied media files, and separate buckets distinguish their processing state. Chroma stores vectors locally without requiring another database server. The Docker configuration pins the S3-compatible storage service.

One orchestrator connects independently testable stages. One of our laptops had no suitable GPU, so we chose CPU as the common setup for the whole group. Models load sequentially to limit RAM use. Acquisition remains optional, allowing supplied data to follow the same route without another download.

Further detail in `docs/data_sources.md`, `docs/p1_requirements.md`.

2. Landing and Formatted

Simple checks before expensive processing

We added the Security Gate to reject obvious risks before decoding or running models. It checks allowed types, byte signatures, file size, image dimensions and archive limits. For example, an executable renamed as a photograph is rejected from its header, while an oversized file stops before costly processing. These are practical input heuristics, not an antivirus scanner. Accepted files enter Temporal Landing and are then copied and verified in Persistent Landing, by modality and species.

The Temporal key combines the first 16 hexadecimal characters of SHA-256 with the filename. SHA-256 reads raw bytes in small blocks without loading a decoder or neural model. This makes it a cheaper first check than computing an embedding, although we have not measured a speed ratio. A hash detects byte identity and supports integrity checks, while an embedding measures similarity in learned features.

Bytes	Name	Landing result
Same	Same	Reuse the existing, verified record
Same	Different	Ingest a separate submitted-file record
Different	Same	Use a new hash and preserve the earlier file
Different	Different	Ingest a new record

We debated using only the content hash. Keeping the name also preserves the identity of each submitted file, so renamed copies remain traceable at ingestion. We accepted the extra storage and left content duplicate removal to Trusted. A filename is not proof of a different source. The shortened key can collide, so copy verification uses the full SHA-256, metadata and byte length.

A completion record is written only after copy verification. If a file has left Temporal, a rerun checks that record and its Persistent target before skipping it. Missing targets or conflicting species/source metadata cause an error. Temporal files are retained by default, allowing interrupted copies to recover. This record confirms Landing completion, not Trusted acceptance.

Formats chosen for our wildlife data

Images become RGB JPEG at quality 95. Photographs contain colour gradients and texture, which JPEG usually stores more compactly than lossless PNG. We chose quality 95 to limit compression artefacts while reducing storage for browsing and retrieval. Dimensions and EXIF orientation are kept without rotating or cropping, and transparency becomes white. The original remains available for uses that require lossless detail.

MPO contains several photographs, even when named `.jpg`. We choose frame 0 as a repeatable single-image view, keeping one indexed asset per source. This avoids giving a multi-frame submission extra weight and keeps provenance simple. It is a selection rule rather than duplicate detection. Six files yield six JPEGs, while all eight additional frames remain in the Persistent originals for later frame-level processing if needed.

After listening to the recordings, we found that rain, wind and other animals commonly overlap with the target call. We chose FLAC PCM16 to avoid another lossy audio codec and retain that sound. FLAC compresses the resulting PCM samples losslessly, usually saving space over uncompressed PCM. Rate, channels and quiet intervals are preserved. Conversion to 16-bit can reduce precision, and FLAC cannot recover earlier MP3 losses. Model-specific resampling happens later.

Text becomes UTF-8 with NFC characters and LF endings, retaining case and punctuation. For JSON, we first look for content fields such as `text`, `body` or `description`, preferring those closest to the root and using a fixed priority for ties. Only when these fields are absent do we select the longest non-empty string, favouring a substantial description over a short title or identifier. This fallback supports unfamiliar schemas. We record the extraction method and JSON path so the choice can be checked against the preserved original. Arrays are handled per record.

Further detail in `docs/landing_zone_documentation.md`, `docs/formatted_zone_documentation.md`.

3. Trusted decisions and quality reports

Trusted checks usability and provenance before choosing a search model. Policies live in YAML so that the rules and their versions can be inspected. A technical failure can reject a file. A warning records uncertainty while keeping potentially useful material.

Type	Checks and decisions	Reason
Images	Decoding and provenance. Limits of 20 MB and 40 MP, with at least 128 pixels per side. Exact hashes, then pHash candidates checked with structure/colour.	Compressed size alone cannot bound decoding memory. Very small images have little usable detail. Duplicate confirmation is stricter than a perceptual hash alone.
Audio	Decoding and provenance, 200 MB cap, duration and amplitude, exact PCM duplicates.	Recordings can be longer than image files. Quiet or distant calls may still be useful, so low energy produces a warning.
Text	Strict decoding and provenance, 4 MB cap, exact duplicates, near-duplicate warnings and privacy-pattern counts.	Species descriptions should remain manageable. Short GBIF records and long articles need different interpretation, not an arbitrary common length.

These limits use decimal MB and set operational budgets for our local setup. Pixel limits also bound image dimensions. Audio memory depends on duration and decoding as well as bytes, so longer recordings still need care. Larger deployments should revise these budgets using measured resource use.

Darkness, blur, monochrome images and possible watermarks produce warnings, preserving imperfect but useful field observations. Watermark checks use Tesseract OCR when the selected batch has at most 1,000 images and Tesseract is installed. Larger batches or unavailable OCR use a lighter corner-edge detector. An OCR error or ten-second timeout also triggers that fallback. This bounds OCR work without requiring it on every machine. Reports record the method and flag candidates for review, not confirmed watermarks. Audio keeps background sound and quiet intervals to preserve field context.

A later failure or rejection keeps previously accepted bytes and records the changed status. Reconciliation exposes outdated index references instead of silently deleting data.

Reports explain the decision for each file

Each check produces JSON under `trusted-zone/_quality/<modality>/`, including the source, policy, decision, reasons and measurements. Report generators aggregate the current stored records, including earlier runs. They do not decode every file again or change acceptance decisions.

The local output includes HTML for review, JSON summaries for automated checks, CSV measurements for analysis and PNG charts. Image size, brightness and sharpness reveal decoding costs and difficult photographs. Audio duration, RMS/peak levels, clipping and low-energy windows expose long, faint or near-saturated recordings. Text length and vocabulary ratios reveal short descriptions, repetition and uneven document sizes. Privacy checks count patterns without publishing matched values. Before/after species and source distributions show filtering effects, while source-field completeness checks traceability. The Trusted guide defines these measures and their interpretation.

What the reference reports show

All 198 reference files remain accepted. The image report used OCR on all 100 images, flagging 32 possible watermarks and 14 monochrome-like images for review. Audio has nine low-energy and two low-RMS flags. Its median duration is 40.91 seconds and its longest recording is 752.37 seconds. Text has a median of 208.5 words, ranging from 33 to 13,869. All records have the required source fields. Species distributions remain unchanged, including the missing lizard audio.

These results describe a curated batch. Synthetic malformed files and fault tests exercise rejection separately. We still need labelled examples to measure watermark and duplicate-detection accuracy. Pairwise text comparison will also need a more efficient candidate index as the dataset grows.

Further detail in `docs/trusted_zone_documentation.md` and reference summaries in `docs/evidence/quality_*.json`.

4. Exploitation and model choices

Exploitation represents accepted Trusted content as vectors called embeddings. These support similarity search across five Chroma collections.

Collection	Model family	Dimensions	Vectors
Images	CLIP	512	100
Audio	CLAP	512	357
Text	Multilingual E5	384	547
Image and text	CLIP-aligned	512	1,763
Audio and text	CLAP	512	3,254
Total			6,021

Why five collections?

Five collections use three trained spaces, CLIP, CLAP and E5, through four model checkpoints including the multilingual CLIP adapter. Constraint 7 requires separate collections by modality for Task 1. Constraint 13 asks for new mixed collections for Task 2. The 100 image and 357 audio vectors therefore appear in both their single-type and shared collections. This costs extra storage but keeps the tasks independently testable, reusing the same models.

Models selected for the tasks and our hardware

CLIP ViT-B/32 provides image and text-to-image retrieval with a manageable base-size encoder. Its processor resizes and centre-crops to 224×224 pixels in memory, leaving Trusted files intact.[1] The multilingual adapter aligns Spanish and English descriptions with those image vectors. CLAP HTSAT-unfused supplies the separate audio/text relationship needed for animal calls.

E5-small specialises in text retrieval with `query:` and `passage:` prefixes. CLIP and CLAP text vectors align with images or sounds, so E5 supplies a separate text-focused route. These choices fit our CPU setup. Comparing their wildlife accuracy would need a labelled benchmark. Model revisions are fixed in `config/model_manifest.json`.

Why 6,021 vectors for only 198 files?

Encoders have limited input lengths. E5 uses chunks of 320 tokens with 40-token overlap, image/text uses 96/12 and audio/text 60/8, matching their configured text encoders. Overlap helps retain context across boundaries at the cost of extra vectors.

Audio uses ten-second windows at 48 kHz mono, keeping local calls searchable within long recordings. Trusted retains the original rate and channels for other uses. Text also appears in shared collections, adding vectors for each model's fragments. Results group segments back into files.

Persistent, incremental indexing

Vectors remain on disk in `data/chromadb/`. Metadata links each part to its source file, model/policy version and position. An unchanged file with all expected parts is skipped on a rerun. Source checksums, policy fingerprints and complete part sets determine reuse. A matching total count alone is insufficient. Interrupted publication can be repaired by processing the incomplete asset again.

Sequential CPU model loading and bounded batches limit memory use. Separate collection runs can repeat compatible inference during the first build. Searches use the verified query copy described in the manual, protecting the original index from Chroma's internal writes. Refreshing that copy requires no inference.

Further detail in `docs/exploitation_zone_documentation.md`, `docs/data_sources.md`, `config/exploitation.yaml`.

[1] The linked CLIP processor configuration fixes the model revision used for this resize and crop.

5. Similarity search and its limits

Cosine measures direction in one vector space

For two non-zero vectors, cosine similarity is

$$s(q, x) = \frac{q \cdot x}{\|q\|\|x\|}.$$

We normalise both vectors, so this becomes a dot product. Values range from -1 to 1, with larger values meaning closer directions. Chroma returns cosine distance, so we display $1 -$ distance. We reject non-finite or incompatible vectors rather than returning misleading scores.

For an illustrative query $q = (1, 0)$, the unit vectors $(0.96, 0.28)$, $(0.6, 0.8)$ and $(0, 1)$ score 0.96, 0.60 and 0.00. The first is closest in this example. These numbers are not probabilities that two animals share a species. Real scores depend on the model and query type, so there is no universal "correct match" threshold in our interface.

Exact search for small sets and HNSW for larger sets

The application counts eligible vectors after filters. With 64 or fewer, it reads the vectors and compares every one. With 65 or more, it asks Chroma's HNSW index. HNSW, or Hierarchical Navigable Small World, is a graph search method that navigates from coarse graph levels to nearby candidates instead of comparing every vector. It is approximate, so it can miss an exact nearest neighbour.

Why choose 64? We wanted a small, bounded exact scan that also works for tiny or new Chroma segments without a persisted HNSW reader. At 512 dimensions, 64 dot products involve 32,768 vector components. The float32 vector payload alone is 128 KiB, excluding metadata, Python objects and normalisation work. We use 64 as a conservative engineering budget, with tests on both sides of the boundary, including filtered collections. A future timing and recall study could tune it for larger installations. The cap counts vector parts, not returned files and not vector dimensions.

For larger sets, we expand the retrieved candidate pool when many matches belong to one file. We then keep distinct assets, so a long recording cannot fill the result list merely because it has many segments. Top-k means the maximum number of returned files, with a default of eight, selectable from 1 to 50 in the interface or with `--limit N`. Filters or limited data can produce fewer results.

Different modalities need a shared trained space

An image and a sentence can be compared when their encoders share a trained CLIP space. The same applies to CLAP audio and text. E5 lives in its own space. Even though CLIP and CLAP both produce 512 numbers, their coordinates mean different things. Computing their cosine would produce a number without a valid similarity interpretation, so we keep those collections separate.

When a query uses several spaces, reciprocal rank fusion (RRF) combines positions rather than raw cosine values. Each file receives $1/(60+r)$ from each ranking where it occurs at position r, counted once per ranking. The constant 60 smooths the advantage of the first few ranks. It is separate from top-k and has not been tuned on labelled wildlife queries. A visible modality allocation can then keep mixed results useful. This output is a combined ranking, not one global cosine ranking.

We therefore distinguish correct search mechanics from useful biological retrieval. Tests can verify filtering, grouping and score handling. Expert relevance labels are still needed to measure recall, ranking quality and species-related errors.

Further detail in `multimodal_tasks/search.py`, `docs/multimodal_tasks_documentation.md`.

6. Tasks 1 and 2, queries and interface

Task 1 / find similar material of the same type

A photograph retrieves photographs, a recording retrieves recordings, and text uses E5 to retrieve descriptions. For a stored record, we reuse its vector and exclude that entire file from the results, including its other segments. For an upload, we prepare the bytes in memory and use the same encoder and vector normalisation as the indexed data.

This supports questions such as "Which stored photographs look like this one?" A high-ranking image may share colour, pose or background without showing the same species. The result cards expose the source and score so that the user can inspect the candidates rather than accept a biological label automatically.

Task 2 / search across types of material

The image/text and audio/text collections each contain both types of data. A text phrase can retrieve photographs and descriptions through CLIP, or recordings and descriptions through CLAP. Image and audio queries can also retrieve mixed results in their compatible spaces. When searching all spaces, RRF combines the rankings as explained on page 5.

We group results by file and allocate available places across modalities. This helps a user see related evidence without a large number of text chunks taking every place. The trade-off is that the final order serves mixed exploration rather than pure cosine ranking. A heavily filtered search can still return only one modality.

Queries reuse processing without entering the dataset

We debated whether uploads should pass every dataset check. After the supervisor's clarification, we chose comparable preparation while giving users more freedom over quality. Queries reuse the Formatted code for RGB JPEG quality 95, FLAC PCM16 and canonical text, followed by the same encoder preparation as indexing. Stored Trusted JPEGs bypass another JPEG encode. This reduces avoidable differences but cannot undo earlier compression or guarantee identical vectors from separately encoded files.

A field recording or photograph may be poor but still carry useful information. Shared Trusted measurements therefore report blur, darkness, small size or quiet audio as warnings. We omit duplicate rejection because a user may be searching for an existing file, and omit the uncalibrated watermark rule. Hard guards still reject invalid inputs, uploads above 10 MiB, images above 20 MP and audio above 30 seconds or eight channels before conversion. These tighter budgets bound interactive CPU work rather than define biological quality.

Uploads stay in memory and are not added to MinIO or Chroma. The response includes an input hash and preparation details for traceability. Quality warnings are visible to the user and stay outside the RAG prompt because a blur measurement is not evidence about an animal.

A local interface for all three Tasks

Tab	What the user can try
Explorar por texto	A text query in E5, either shared space or all spaces, with species filters and result quantity.
Buscar similares	An image or short recording for same-type or mixed retrieval, with related-file actions.
Preguntar al corpus / RAG	A question with an optional image, with cited claims, candidate species, quality notes and response status.

Here, "corpus" simply means the stored collection of documents and media. The static frontend needs no separate build tool. The Python server binds to the local machine, checks host/origin and serialises CPU requests. This makes classroom setup simpler. A public multiuser service would need authentication and a different deployment design.

Further detail in `docs/user_guide.md`, `docs/multimodal_tasks_documentation.md`.

7. Task 3 / answers from retrieved evidence

Task 3 uses retrieval augmented generation, or RAG. It retrieves stored information and gives selected passages to a language model, allowing answers to cite sources or decline questions with insufficient evidence.

How a question with an image is answered

The image first receives the shared preparation on page 6. BLIP produces a caption, which passes a wildlife vocabulary check before retrieval. CLIP then searches the image/text collection for visual candidates. Their species names guide E5 searches for the requested subject, such as habitat or diet, with a bounded lexical search for explicit terms. Global text searches keep one candidate from deciding the answer. Without an image, the question passes the vocabulary check and uses E5 plus multilingual CLIP text retrieval. Appendix A shows both routes.

The selector keeps at most eight literal units and 1,600 characters to bound prompt size and CPU cost. Full references remain visible. The prompt requires evidence-based claims and uncertainty about the photographed animal. We avoid repeating a noisy caption in every biological search.

Qwen receives text, not image pixels. BLIP is the image-to-text step. Its caption can be wrong. In the recorded frog example it described "a frog with a dumb barbell". We treat this as a model observation, never as a verified identification. Similar frog references can still support a conditional answer about the retrieved candidates' habitat or diet.

Why Qwen and a separate captioning model?

We use Qwen2.5-3B-Instruct in Q4_K_M GGUF form through llama.cpp. Quantisation reduces the model's memory needs, making CPU execution possible without a paid API. The model file is pinned and checked by SHA-256. BLIP gives us a separate image captioning step that we can inspect and test. This design loses some visual detail compared with a model that directly reads images, but keeps the components manageable on our machine. We have not established superiority over larger models or native visual assistants.

Generation uses a JSON grammar, then checks the response structure, allowed evidence IDs and exact quote matches. Multi-aspect questions can produce up to two claims. If one aspect has no support, it can remain empty and the interface reports a partial answer. These checks catch invented reference IDs and quotes. They do not prove that the model interpreted a passage correctly.

Failure handling and what we have validated

We decided to bound generation to 320 tokens, 120 seconds and four CPU threads, with a 2.5 GiB available-RAM check. These choices keep requests manageable on our laptops. The vocabulary gate also avoids spending generation time on unrelated questions, such as cooking advice, which our wildlife sources cannot support. It checks the question without an image, or the BLIP caption with one, and may reject valid wording. This is a domain heuristic, not a reliable detector of every unfamiliar input.

An evidence-based abstention differs from invalid JSON, a timeout or an incomplete output. The interface exposes the status and retains any retrieved references. Both the gate and the subject-selection rules need broader language evaluation.

Earlier native checks exercised text habitat, stored-image habitat, conditional frog identification, an unsupported genome question and an unrelated request. A separate frog habitat/diet browser run took 131.8 seconds overall, including 76.1 seconds of generation. One lynx answer was less directly relevant. These runs show that the system can execute the workflow, but they are not an accuracy benchmark. The later query-preparation revision has offline regression tests, but those historical native runs were not repeated for it.

Further evaluation needs labelled external queries to measure relevant retrieval, unsupported answers and unnecessary abstentions.

Further detail in `docs/generative_tasks_documentation.md`, `docs/evidence/native_models.json`. Model cards Qwen GGUF and BLIP.

8. Orchestration, validation and reproducibility

The 16 steps, in execution order

Stage	Numbered steps
Storage	1 Check connection, 2 create missing buckets
Landing	3 Ingest local raw files, 4 copy and verify in Persistent
Formatted	5 Format images, 6 audio, 7 text
Trusted	8 Validate images, 9 audio, 10 text
Exploitation	11 Index images, 12 text, 13 audio, 14 image/text, 15 audio/text
Verification	16 Reconcile accepted files and vector parts

Stages run sequentially, with optional acquisition adding three downloads after storage setup. Subprocesses use the active Python interpreter, timeouts and UTF-8 logs. Atomic JSON reports record progress. Failures stop later stages and reruns retry incomplete work after checking stored state. A local lock prevents two orchestrators in one checkout, but cannot coordinate manual writers or other machines. There is no transaction across all zones.

Tests target behaviour and failure recovery

On 4 November, 469 offline tests passed with 72.37% statement coverage, using generated files, storage doubles, temporary real Chroma and local HTTP. Cases cover conversion, provenance, duplicates, query limits, citations and idempotence. Recovery tests check completion records, conflicting sources, interrupted storage, preserved outputs, rejected Trusted records and missing/stale vector parts. Search tests include the 64/65 boundary.

Coverage shows which statements ran, not accuracy or complete outcome checks. We kept the same measurement scope. Some paths require storage/model checks and others remain untested. Executing lines merely to reach 100% would not establish better behaviour.

Isolated S3/Chroma integration also passed on 4 November, including a rerun with deterministic encoders. Earlier CPU runs produced seven vectors from three synthetic assets, then zero new vectors. Earlier preservation checks compared original Chroma hashes/counts and S3 keys, sizes and ETags without rebuilding the reference. Native retrieval and RAG evidence is separately dated.

Automatic checks and reproducibility limits

GitHub Actions runs after pushes and pull requests. Windows/Linux jobs install the same requirements, check style and run offline tests with a 60% coverage floor. Linux also tests isolated S3 storage. That floor detects large regressions, not overall quality. All three jobs passed for commit `d470f07`.[2] JUnit and coverage reports are retained as artifacts.

A clone reproduces the tests and synthetic demonstration. The 198-file reference also requires the data handoff, metadata and pinned model cache, since new downloads may change. Query copies become stale, long audio can use substantial RAM and text duplicate comparisons are quadratic. We have no labelled retrieval or biological benchmark. Our priority was an inspectable, recoverable implementation with evidence for its guarantees.

A possible extension beyond P1

VL-JEPA is future work beyond P1. It predicts text embeddings and decodes words when needed. Its authors report 2.85 times fewer decoding operations in their experiment.[3] We could test camera-trap video retrieval before generating explanations, with new compatible vectors, wildlife relevance tests and CPU/RAM measurements. Source retrieval and citation checks would remain necessary.

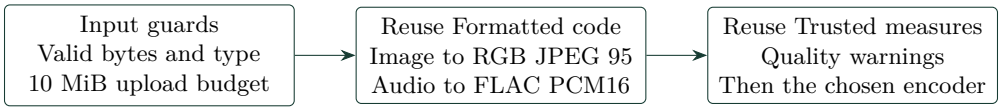
[2] This identifier names the tested repository version. Actions run 41820793368 passed on 4 November 2026. Check the submitted revision after each push.

[3] Research reference, not an EcoSentinel result. Delong Chen and colleagues, *VL-JEPA: Joint Embedding Predictive Architecture for Vision-language*, arXiv:2512.10942v2, revised 2 February 2026.

Appendix A / Query routes

All search arrows below read the verified Chroma copy in `data/search_index/`. Models load from the local cache. Query uploads stay in memory and do not enter the stored dataset.

Shared preparation for uploaded media



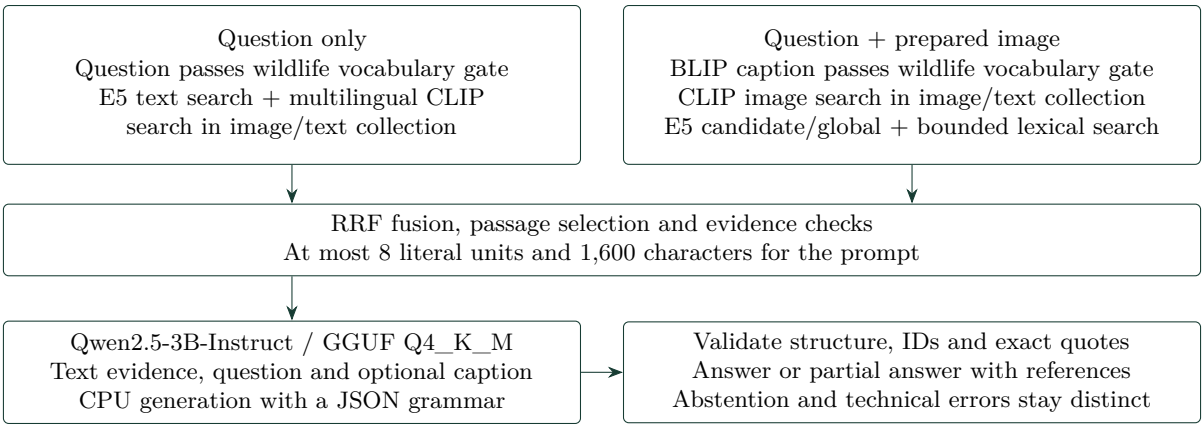
Images also have a 20 MP guard, audio a 30-second/eight-channel guard. No duplicate or watermark rejection is applied to queries. Stored Trusted JPEGs skip recompression. Text uses canonical Unicode. Audio encoding adds 48 kHz mono preparation and ten-second windows.

Tasks 1 and 2 / Input, encoder and searched collection

Input	Encoder	Task 1, same type	Task 2, shared types
Image	CLIP ViT-B/32	Images	Image + text
Audio	CLAP HTSAT-unfused	Audio	Audio + text
Text	Multilingual E5-small	Text	Text route in all-space search
Text	Multilingual CLIP adapter	—	Image + text
Text	CLAP text encoder	—	Audio + text

Each row means input → encoder → compatible Chroma collection. Stored-record searches reuse an existing vector and exclude its source file. All-space text search fuses three rankings with RRF, while audio fuses its window rankings. Results are grouped by file and mixed searches can allocate places by modality. Image and audio vectors are never compared directly.

Task 3 / Question with or without an image



Failed domain or evidence checks stop before Qwen. BLIP is `Salesforce/blip-image-captioning-base`. Qwen receives text, not pixels, and quality warnings stay outside its prompt. Task 3 accepts text and optional images, not audio.

Where data is read

Vector search and evidence passages come from the Chroma query copy. Media previews/downloads and a stored image selected for RAG read accepted files from Trusted S3 storage after source checks. Upload queries write to neither store. Cached weights avoid inference-time downloads. Persistent Landing and Formatted buckets are not part of the query route.

Routes checked against `multimodal_tasks/query_preparation.py`, `search.py`, `rag.py` in the same package, and `webapp/server.py`. Model revisions are fixed in the embedding manifest and RAG configuration.